

PROJECT BASED EXPERIMENTAL LEARNING

COURSE: DATABASE MANAGEMENT SYSTEM (NOSQL / MONGODB)

Project Report: MongoDB Student Database Operations

SUBMITTED TO
Mr. Harendra Singh Rajpoot

INDUSTRY PARTNER
IBM

SUBMITTED BY
Krishna Kant Dubey

CLASS & SECTION
BCADS27 (3rd Semester)

UNIVERSITY ROLL NO.
1250258245

ACADEMIC YEAR
Current Session

Babu Banarasi Das University
Lucknow, Uttar Pradesh

ACKNOWLEDGMENT

I would like to express my special thanks of gratitude to my mentor **Mr. Harendra Singh Rajpoot**, who gave me the golden opportunity to do this wonderful project on MongoDB Student Database Operations. My mentor helped me a lot in completing this project within the given time frame, and I learned many new things about NoSQL databases from him. He converted his experience into our knowledge.

Secondly, I want to show my gratitude to **IBM**, which provided me this project-based learning experience; it helped me to achieve my goals more efficiently.

ABSTRACT

This report presents a MongoDB-based Student Database Management project that demonstrates the core Create, Read, Update and Delete (CRUD) operations of a NoSQL document database. A "students" collection is used to store student records containing roll number, name, age, department, and marks. The project implements single and bulk insertion, conditional updates, single and bulk deletion, and a wide range of query operations using MongoDB comparison and logical operators such as \$gt, \$lt, \$eq, \$gte, \$ne, \$and, \$or, and \$not. The implementation demonstrates how MongoDB can be used to efficiently manage, filter, and retrieve structured student data.

INTRODUCTION

MongoDB is a popular NoSQL, document-oriented database that stores data as flexible, JSON-like documents rather than in rigid rows and columns. This makes it well suited for applications where the data structure may evolve over time, such as academic record-keeping systems. This project focuses on building and querying a "students" collection that stores academic details of 51 students, and demonstrates the standard database operations—insertion, updation, deletion, and querying—that form the backbone of any real-world database-driven application.

Methodology Overview

The project was carried out step by step inside the MongoDB shell (mongosh), with every operation executed against a single collection named "students" in a dedicated database. The complete methodology, along with exact code queries and outputs, is detailed in the sections that follow.

DATABASE OPERATIONS & CODE

1. Insert One

Inserts a single student document into the students collection.

```
db.students.insertOne({
  rollNo: 101,
  name: "Haruto Sato",
  age: 20,
  department: "CSE",
  marks: 85
})
```

2. Insert Many (Roll Nos. 102 – 151)

Inserts multiple student documents into the collection simultaneously using a dataset of Japanese student names.

```
db.students.insertMany([
  { rollNo: 102, name: "Yuki Suzuki", age: 19, department: "IT", marks: 78 },
  { rollNo: 103, name: "Ren Takahashi", age: 21, department: "CSE", marks: 92 },
  { rollNo: 104, name: "Sora Tanaka", age: 20, department: "ECE", marks: 81 },
  { rollNo: 105, name: "Kaito Ito", age: 22, department: "ME", marks: 74 },
  { rollNo: 106, name: "Riku Watanabe", age: 19, department: "CSE", marks: 88 },
  { rollNo: 107, name: "Hiroshi Yamamoto", age: 20, department: "IT", marks: 69 },
  { rollNo: 108, name: "Takumi Nakamura", age: 21, department: "ECE", marks: 95 },
  { rollNo: 109, name: "Daiki Kobayashi", age: 20, department: "CSE", marks: 83 },
  { rollNo: 110, name: "Sakura Kato", age: 19, department: "IT", marks: 76 },
  { rollNo: 111, name: "Kenji Yoshida", age: 22, department: "ME", marks: 72 },
  { rollNo: 112, name: "Yuto Yamada", age: 20, department: "CSE", marks: 89 },
  { rollNo: 113, name: "Akira Sasaki", age: 21, department: "ECE", marks: 67 },
  { rollNo: 114, name: "Hayato Yamaguchi", age: 19, department: "IT", marks: 91 },
  { rollNo: 115, name: "Ryota Matsumoto", age: 20, department: "CSE", marks: 79 },
  { rollNo: 116, name: "Shota Inoue", age: 21, department: "ECE", marks: 84 },
  { rollNo: 117, name: "Kazuki Kimura", age: 22, department: "ME", marks: 73 },
  { rollNo: 118, name: "Yuma Hayashi", age: 20, department: "CSE", marks: 96 },
  { rollNo: 119, name: "Naoki Shimizu", age: 19, department: "IT", marks: 68 },
  { rollNo: 120, name: "Satoshi Yamazaki", age: 21, department: "ECE", marks: 87 },
  { rollNo: 121, name: "Taro Mori", age: 20, department: "CSE", marks: 75 },
  { rollNo: 122, name: "Hana Abe", age: 19, department: "IT", marks: 90 },
  { rollNo: 123, name: "Aiko Ikeda", age: 22, department: "ME", marks: 71 },
  { rollNo: 124, name: "Emi Hashimoto", age: 20, department: "CSE", marks: 86 },
  { rollNo: 125, name: "Yui Ishikawa", age: 21, department: "ECE", marks: 80 },
  { rollNo: 126, name: "Mei Ogawa", age: 19, department: "IT", marks: 94 },
  // ...continued on next page
])
```

2. Insert Many (Continued: Roll Nos. 127 – 151)

```
{ rollNo: 127, name: "Rin Goto", age: 20, department: "CSE", marks: 77 },
{ rollNo: 128, name: "Kenta Okada", age: 21, department: "ECE", marks: 82 },
{ rollNo: 129, name: "Tatsuya Hasegawa", age: 22, department: "ME", marks: 65 },
{ rollNo: 130, name: "Masaki Murakami", age: 20, department: "CSE", marks: 93 },
{ rollNo: 131, name: "Shun Kondo", age: 19, department: "IT", marks: 70 },
{ rollNo: 132, name: "Koji Ishii", age: 21, department: "ECE", marks: 89 },
{ rollNo: 133, name: "Daichi Saito", age: 20, department: "CSE", marks: 81 },
{ rollNo: 134, name: "Yusuke Sakamoto", age: 19, department: "IT", marks: 85 },
{ rollNo: 135, name: "Tomoya Endo", age: 22, department: "ME", marks: 78 },
{ rollNo: 136, name: "Minato Aoki", age: 20, department: "CSE", marks: 97 },
{ rollNo: 137, name: "Asahi Fujita", age: 21, department: "ECE", marks: 74 },
{ rollNo: 138, name: "Keita Nishimura", age: 19, department: "IT", marks: 88 },
{ rollNo: 139, name: "Ayumi Fukuda", age: 20, department: "CSE", marks: 66 },
{ rollNo: 140, name: "Miyu Ota", age: 21, department: "ECE", marks: 92 },
{ rollNo: 141, name: "Nanami Miura", age: 22, department: "ME", marks: 69 },
{ rollNo: 142, name: "Ichiro Fujii", age: 20, department: "CSE", marks: 84 },
{ rollNo: 143, name: "Jiro Okamoto", age: 19, department: "IT", marks: 79 },
{ rollNo: 144, name: "Shinji Matsuda", age: 21, department: "ECE", marks: 91 },
{ rollNo: 145, name: "Hikaru Nakagawa", age: 20, department: "CSE", marks: 87 },
{ rollNo: 146, name: "Makoto Nakano", age: 19, department: "IT", marks: 73 },
{ rollNo: 147, name: "Junpei Harada", age: 21, department: "ECE", marks: 86 },
{ rollNo: 148, name: "Toru Ono", age: 22, department: "ME", marks: 68 },
{ rollNo: 149, name: "Kaori Takeda", age: 20, department: "CSE", marks: 90 },
{ rollNo: 150, name: "Rika Kaneko", age: 21, department: "IT", marks: 76 },
{ rollNo: 151, name: "Eita Kudo", age: 19, department: "ECE", marks: 83 }
]
```

3. Update One

Updates the marks of roll number 101 (Haruto Sato) from 85 to 90.

```
db.students.updateOne(
  { rollNo: 101 },
  { $set: { marks: 90 } }
)
```

Output: { acknowledged: true, matchedCount: 1, modifiedCount: 1 }

4. Update Many

Updates all CSE department documents to add a new status field set to "Active".

```
db.students.updateMany(
  { department: "CSE" },
  { $set: { status: "Active" } }
)
```

Output: { acknowledged: true, matchedCount: 16, modifiedCount: 16 }

5. Delete One

Deletes the document matching roll number 151 (Eita Kudo).

```
db.students.deleteOne({ rollNo: 151 })
```

```
Output: { acknowledged: true, deletedCount: 1 }
```

6. Delete Many

Deletes all documents where marks are less than 40.

```
db.students.deleteMany({ marks: { $lt: 40 } })
```

```
Output: { acknowledged: true, deletedCount: 0 }
```

7. QUERY OPERATIONS

A series of find() queries executed using comparison and logical operators.

Marks greater than 80:

```
db.students.find({ marks: { $gt: 80 } })
```

103 Ren Takahashi 21 CSE 92	104 Sora Tanaka 20 ECE 81	106 Riku Watanabe 19 CSE 88
108 Takumi Nakamura 21 ECE 95	109 Daiki Kobayashi 20 CSE 83	112 Yuto Yamada 20 CSE 89
114 Hayato Yamaguchi 19 IT 91	116 Shota Inoue 21 ECE 84	118 Yuma Hayashi 20 CSE 96
120 Satoshi Yamazaki 21 ECE 87	122 Hana Abe 19 IT 90	124 Emi Hashimoto 20 CSE 86
126 Mei Ogawa 19 IT 94	128 Kenta Okada 21 ECE 82	130 Masaki Murakami 20 CSE 93
132 Koji Ishii 21 ECE 89	133 Daichi Saito 20 CSE 81	134 Yusuke Sakamoto 19 IT 85
136 Minato Aoki 20 CSE 97	138 Keita Nishimura 19 IT 88	140 Miyu Ota 21 ECE 92
142 Ichiro Fujii 20 CSE 84	144 Shinji Matsuda 21 ECE 91	145 Hikaru Nakagawa 20 CSE 87
147 Junpei Harada 21 ECE 86	149 Kaori Takeda 20 CSE 90	

Marks less than 80:

```
db.students.find({ marks: { $lt: 80 } })
```

102 Yuki Suzuki 19 IT 78	105 Kaito Ito 22 ME 74	107 Hiroshi Yamamoto 20 IT 69
110 Sakura Kato 19 IT 76	111 Kenji Yoshida 22 ME 72	113 Akira Sasaki 21 ECE 67
115 Ryota Matsumoto 20 CSE 79	117 Kazuki Kimura 22 ME 73	119 Naoki Shimizu 19 IT 68
121 Taro Mori 20 CSE 75	123 Aiko Ikeda 22 ME 71	127 Rin Goto 20 CSE 77
129 Tatsuya Hasegawa 22 ME 65	131 Shun Kondo 19 IT 70	135 Tomoya Endo 22 ME 78
137 Asahi Fujita 21 ECE 74	139 Ayumi Fukuda 20 CSE 66	141 Nanami Miura 22 ME 69
143 Jiro Okamoto 19 IT 79	146 Makoto Nakano 19 IT 73	148 Toru Ono 22 ME 68

Marks equal to 85:

```
db.students.find({ marks: { $eq: 85 } })
```

101 Haruto Sato 20 CSE 85	134 Yusuke Sakamoto 19 IT 85
---------------------------	------------------------------

Marks greater than or equal to 80:

```
db.students.find({ marks: { $gte: 80 } })
```

101 Haruto Sato 20 CSE 90	103 Ren Takahashi 21 CSE 92	104 Sora Tanaka 20 ECE 81
106 Riku Watanabe 19 CSE 88	108 Takumi Nakamura 21 ECE 95	109 Daiki Kobayashi 20 CSE 83
112 Yuto Yamada 20 CSE 89	114 Hayato Yamaguchi 19 IT 91	116 Shota Inoue 21 ECE 84
118 Yuma Hayashi 20 CSE 96	120 Satoshi Yamazaki 21 ECE 87	122 Hana Abe 19 IT 90
124 Emi Hashimoto 20 CSE 86	125 Yui Ishikawa 21 ECE 80	126 Mei Ogawa 19 IT 94
128 Kenta Okada 21 ECE 82	130 Masaki Murakami 20 CSE 93	132 Koji Ishii 21 ECE 89
133 Daichi Saito 20 CSE 81	134 Yusuke Sakamoto 19 IT 85	136 Minato Aoki 20 CSE 97
138 Keita Nishimura 19 IT 88	140 Miyu Ota 21 ECE 92	142 Ichiro Fujii 20 CSE 84
144 Shinji Matsuda 21 ECE 91	145 Hikaru Nakagawa 20 CSE 87	147 Junpei Harada 21 ECE 86
149 Kaori Takeda 20 CSE 90		

Marks not equal to 85:

```
db.students.find({ marks: { $ne: 85 } })
```

Returns all records in the collection excluding documents with marks equal to 85.

Marks greater than 70 AND age less than 23:

```
db.students.find({
  $and: [
    { marks: { $gt: 70 } },
    { age: { $lt: 23 } }
  ]
})
```

101 Haruto Sato 20 CSE 90	102 Yuki Suzuki 19 IT 78	103 Ren Takahashi 21 CSE 92
104 Sora Tanaka 20 ECE 81	105 Kaito Ito 22 ME 74	106 Riku Watanabe 19 CSE 88
108 Takumi Nakamura 21 ECE 95	109 Daiki Kobayashi 20 CSE 83	110 Sakura Kato 19 IT 76
111 Kenji Yoshida 22 ME 72	112 Yuto Yamada 20 CSE 89	114 Hayato Yamaguchi 19 IT 91
115 Ryota Matsumoto 20 CSE 79	116 Shota Inoue 21 ECE 84	117 Kazuki Kimura 22 ME 73
118 Yuma Hayashi 20 CSE 96	120 Satoshi Yamazaki 21 ECE 87	121 Taro Mori 20 CSE 75
122 Hana Abe 19 IT 90	123 Aiko Ikeda 22 ME 71	124 Emi Hashimoto 20 CSE 86
125 Yui Ishikawa 21 ECE 80	126 Mei Ogawa 19 IT 94	127 Rin Goto 20 CSE 77
128 Kenta Okada 21 ECE 82	130 Masaki Murakami 20 CSE 93	132 Koji Ishii 21 ECE 89
133 Daichi Saito 20 CSE 81	134 Yusuke Sakamoto 19 IT 85	135 Tomoya Endo 22 ME 78
136 Minato Aoki 20 CSE 97	137 Asahi Fujita 21 ECE 74	138 Keita Nishimura 19 IT 88
140 Miyu Ota 21 ECE 92	142 Ichiro Fujii 20 CSE 84	143 Jiro Okamoto 19 IT 79
144 Shinji Matsuda 21 ECE 91	145 Hikaru Nakagawa 20 CSE 87	146 Makoto Nakano 19 IT 73
147 Junpei Harada 21 ECE 86	149 Kaori Takeda 20 CSE 90	150 Rika Kaneko 21 IT 76

Department is CSE OR ECE:

```
db.students.find({
  $or: [
    { department: "CSE" },
    { department: "ECE" }
  ]
})
```

CSE Records:

101 Haruto Sato	103 Ren Takahashi	106 Riku Watanabe
109 Daiki Kobayashi	112 Yuto Yamada	115 Ryota Matsumoto
118 Yuma Hayashi	121 Taro Mori	124 Emi Hashimoto
127 Rin Goto	130 Masaki Murakami	133 Daichi Saito
136 Minato Aoki	139 Ayumi Fukuda	142 Ichiro Fujii
145 Hikaru Nakagawa	149 Kaori Takeda	

ECE Records:

104 Sora Tanaka	108 Takumi Nakamura	113 Akira Sasaki
116 Shota Inoue	120 Satoshi Yamazaki	125 Yui Ishikawa

128 Kenta Okada	132 Koji Ishii	137 Asahi Fujita
140 Miyu Ota	144 Shinji Matsuda	147 Junpei Harada
151 Eita Kudo		

Marks NOT greater than 80:

<code>db.students.find({ marks: { \$not: { \$gt: 80 } } })</code>		
102 Yuki Suzuki 19 IT 78	105 Kaito Ito 22 ME 74	107 Hiroshi Yamamoto 20 IT 69
110 Sakura Kato 19 IT 76	111 Kenji Yoshida 22 ME 72	113 Akira Sasaki 21 ECE 67
115 Ryota Matsumoto 20 CSE 79	117 Kazuki Kimura 22 ME 73	119 Naoki Shimizu 19 IT 68
121 Taro Mori 20 CSE 75	123 Aiko Ikeda 22 ME 71	125 Yui Ishikawa 21 ECE 80
127 Rin Goto 20 CSE 77	129 Tatsuya Hasegawa 22 ME 65	131 Shun Kondo 19 IT 70
135 Tomoya Endo 22 ME 78	137 Asahi Fujita 21 ECE 74	139 Ayumi Fukuda 20 CSE 66
141 Nanami Miura 22 ME 69	143 Jiro Okamoto 19 IT 79	146 Makoto Nakano 19 IT 73
148 Toru Ono 22 ME 68	150 Rika Kaneko 21 IT 76	

8. MONGODB OPERATOR SUMMARY

Operator	Meaning	Example
\$gt	Greater than	marks: { \$gt: 80 }
\$lt	Less than	marks: { \$lt: 80 }
\$eq	Equal to	marks: { \$eq: 85 }
\$gte	Greater than or equal to	marks: { \$gte: 80 }
\$ne	Not equal to	marks: { \$ne: 85 }
\$and	All conditions must be true	marks > 70 AND age < 23
\$or	At least one condition must be true	CSE OR ECE
\$not	Negates a condition	NOT (marks > 80)

Total dataset: 51 students (Roll No. 101–151).

IMPLEMENTATION DETAILS

The project is implemented using MongoDB and executed through the MongoDB Shell (mongosh):

- A single collection named "students" is used, with fields: rollNo, name, age, department, and marks.
- insertOne() and insertMany() are used to populate the collection with 51 student records.
- updateOne() and updateMany() with the \$set operator are used to modify existing fields and add new ones (e.g., status).
- deleteOne() and deleteMany() are used to remove individual and bulk records based on a filter condition.
- find() is combined with comparison operators (\$gt, \$lt, \$eq, \$gte, \$ne) and logical operators (\$and, \$or, \$not) to retrieve specific subsets of student data.
- Each operation's acknowledgement object (acknowledged, matchedCount, modifiedCount, deletedCount) is used to verify that it executed as intended.

RESULTS

All CRUD operations executed successfully on the students collection:

- The insertOne and insertMany operations populated the collection with 51 student records.
- The updateOne operation successfully changed one student's marks, while updateMany correctly modified all 16 CSE-department documents to add an "Active" status.
- The deleteOne operation removed one record and deleteMany found no records matching its filter, correctly reporting a deletedCount of 0.
- The query operations correctly filtered students by marks and department using comparison and logical operators, demonstrating that MongoDB's query engine returns accurate, condition-based results across a realistic-sized dataset.

CONCLUSION

This project highlights the practical use of MongoDB as a NoSQL database for managing structured academic data. By implementing insertion, update, deletion and a wide variety of query operations on a students collection, the project demonstrates a solid, hands-on understanding of MongoDB's Create, Read, Update and Delete (CRUD) capabilities as well as its comparison and logical query operators. The approach shows how a document database can be used to store and efficiently retrieve real-world tabular data such as student records.

FUTURE SCOPE

- Use the MongoDB Aggregation Framework to compute department-wise averages, toppers and rankings.
- Add schema validation rules to enforce data types and required fields at the database level.
- Create indexes on frequently queried fields (e.g. marks, department) to improve performance.
- Build a front-end / web dashboard (e.g. using Node.js and Express) to interact with the database visually.
- Extend the collection with additional entities such as courses, attendance and results, and model relationships between them.

REFERENCES

- MongoDB Official Documentation: <https://www.mongodb.com/docs/>
- MongoDB CRUD Operations: <https://www.mongodb.com/docs/manual/crud/>
- MongoDB Query Operators: <https://www.mongodb.com/docs/manual/reference/operator/query/>
- mongosh (MongoDB Shell) Documentation: <https://www.mongodb.com/docs/mongodb-shell/>

THANK YOU!

MongoDB Student Database Operations Project Report
Submitted by **Krishna Kant Dubey** (Roll No. **1250258245**)
Babu Banarasi Das University, Lucknow, U.P